

# FDTT 704: K-means Clustering

Dr. Mutua Kilai, PhD

September-December 2026

Department of Pure and Applied Sciences

# Introduction

- The remaining lessons concern methods for reducing the dimension of our observation space ( $n$ ); these methods are commonly referred to as clustering.
- K-means clustering is one of the most commonly used clustering algorithms for partitioning observations into a set of  $k$  groups (i.e.  $k$  clusters), where  $k$  is pre-specified by the analyst.
- k-means, like other clustering algorithms, tries to classify observations into mutually exclusive groups (or clusters), such that observations within the same cluster are as similar as possible (i.e., high intra-class similarity), whereas observations from different clusters are as dissimilar as possible (i.e., low inter-class similarity).

# Pre-requisites

- For this lessons we'll use the following packages (note that the primary function to perform k-means, `kmeans()`, is provided in the stats package that comes with your basic R installation):

```
1 library(dplyr)      # for data manipulation
2 library(ggplot2)    # for data visualization
3 library(stringr)    # for string functionality
4 library(cluster)    # for general clustering algorithms
5 library(factoextra) # for visualizing cluster results
6
7 mnist <- dslabs::read_mnist()
8
9 url <- "https://koalaverse.github.io/homlr/data/my_basket.csv"
10 my_basket <- readr::read_csv(url)
```

# Distance Measures

- The classification of observations into groups requires some method for computing the distance or the (dis)similarity between each pair of observations which form a distance or dissimilarity or matrix.
- There are many approaches to calculating these distances; the choice of distance measure is a critical step in clustering (as it was with KNN).
- It defines how the similarity of two observations ( $x_a$  and  $x_b$  for all  $j$  features) is calculated and it will influence the shape and size of the clusters.
- Euclidean distance (i.e., straight line distance, or as the crow flies) is very sensitive to outliers; when they exist they can skew the cluster results which gives false confidence in the compactness of the cluster.

# Cont'd

- If your features follow an approximate Gaussian distribution then Euclidean distance is a reasonable measure to use.
- However, if your features deviate significantly from normality or if you just want to be more robust to existing outliers, then Manhattan, Minkowski, or Gower distances are often better choices.
- If you are analyzing unscaled data where observations may have large differences in magnitude but similar behavior then a correlation-based distance is preferred.
- It is possible for large volume and low volume customers to exhibit similar behaviors; however, due to their purchasing magnitude the scale of the data may skew the clusters if not using a correlation-based distance measure.

# Defining Clusters

- The basic idea behind k-means clustering is constructing clusters so that the total within-cluster variation is minimized.
- There are several k-means algorithms available for doing this.
- The standard algorithm is the Hartigan-Wong algorithm (Hartigan and Wong 1979), which defines the total within-cluster variation as the sum of the Euclidean distances between observation  $i$ 's feature values and the corresponding centroid:

$$W(C_k) = \sum_{x_i \in C_k} (x_i - \mu_k)^2$$



# Cont'd

- i.  $x_i$  is an observation belonging to the cluster  $C_k$
- Where
  - ii.  $\mu_k$  is the mean value of the points assigned to the cluster  $C_k$
- Each observation ( $x_i$ ) is assigned to a given cluster such that the sum of squared (SS) distances of each observation to their assigned cluster centers ( $\mu_k$ ) is minimized.

# Cont'd

- We define the total within-cluster variation as follows:

$$SS_{within} = \sum_{k=1}^k W(C_k) = \sum_{k=1}^k \sum_{x_i \in C_k} (x_i - \mu_k)^2$$

- The  $SS_{within}$  measures the compactness (i.e., goodness) of the resulting clusters and we want it to be as small as possible.

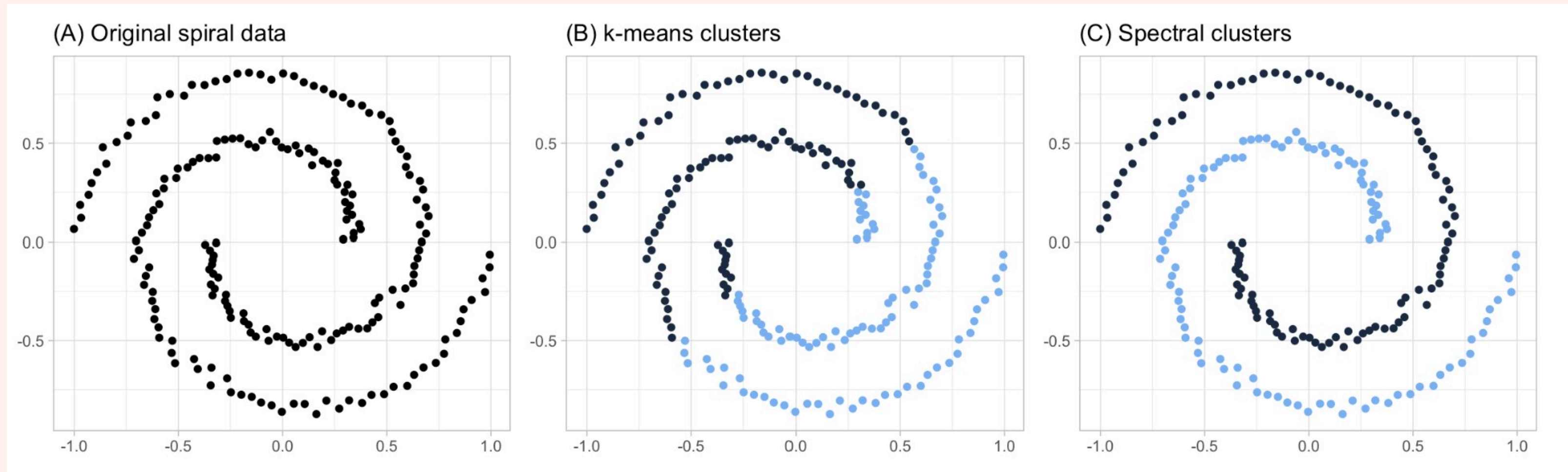


# Cont'd

- The underlying assumptions of k-means requires points to be closer to their own cluster center than to others.

This assumption can be ineffective when the clusters have complicated geometries as k-means requires convex boundaries.

# Cont'd



- The assumptions of k-means leads it ineffective in capturing complex geometric groupings; however, spectral clustering allows you to cluster data that is connected but not necessarily clustered within convex boundaries.

# k-means algorithm

- The first step when using k-means clustering is to indicate the number of clusters ( $k$ ) that will be generated in the final solution.
- Unfortunately, unless our data set is very small, we cannot evaluate every possible cluster combination because there are almost  $k^n$  ways to partition  $n$  observations into  $k$  clusters.
- Next, the algorithm computes the new center (i.e., mean value) of each cluster. The term centroid update is used to define this step.
- Now that the centers have been recalculated, every observation is checked again to see if it might be closer to a different cluster. All the objects are reassigned again using the updated cluster means.

# Summary of k-means algorithm

1. Specify the number of clusters ( $k$ ) to be created (this is done by the analyst).
2. Select  $k$  observations at random from the data set to use as the initial cluster centroids.
3. Assign each observation to their closest centroid based on the distance measure selected.

# Cont'd

4. For each of the  $k$  clusters update the cluster centroid by calculating the new mean values of all the data points in the cluster. The centroid for the  $i$ -th cluster is a vector of length  $p$  containing the means of all  $p$  features for the observations in cluster  $i$ .
5. Iteratively minimize  $SS_{within}$ . That is, iterate steps 3–4 until the cluster assignments stop changing (beyond some threshold) or the maximum number of iterations is reached. A good rule of thumb is to perform 10–20 iterations.

# Determining Optimal Clusters

1. Compute k-means clustering for different values of  $k$ . For instance by varying  $k$  from 1-20 clusters
2. For each  $k$  calculate the total within-cluster sum of squares
3. Plot the curve of within-cluster sum of squares according to the number of clusters  $k$
4. The location of a bend (i.e., elbow) in the plot is generally considered as an indicator of the appropriate number of clusters.



# Cont'd

```
1 features <- mnist$train$images
2
3 # Use k-means model with 10 centers and 10 random starts
4 mnist_clustering <- kmeans(features, centers = 10, nstart = 10)
```

# Cont'd

```

1 # Extract cluster centers
2 mnist_centers <- mnist_clustering$centers
3
4 # Plot typical cluster digits
5 par(mfrow = c(2, 5), mar=c(0.5, 0.5, 0.5, 0.5))
6 layout(matrix(seq_len(nrow(mnist_centers)), 2, 5, byrow = FALSE))
7 for(i in seq_len(nrow(mnist_centers))) {
8   image(matrix(mnist_centers[i, ], 28, 28)[, 28:1],
9           col = gray.colors(12, rev = TRUE), xaxt="n", yaxt="n")
10 }

```





# Cont'd

```

1 # Create mode function
2 mode_fun <- function(x){
3   which.max(tabulate(x))
4 }
5
6 mnist_comparison <- data.frame(
7   cluster = mnist_clustering$cluster,
8   actual = mnist$train$labels
9 ) %>%
10  group_by(cluster) %>%
11  mutate(mode = mode_fun(actual)) %>%
12  ungroup() %>%
13  mutate_all(factor, levels = 0:9)
14
15 # Create confusion matrix and plot results
16 yardstick::conf_mat(
17   mnist_comparison,
18   truth = actual.

```

| Prediction \ Truth | 0    | 1    | 2    | 3    | 4    | 5    | 6    | 7    | 8    | 9    |
|--------------------|------|------|------|------|------|------|------|------|------|------|
| 0                  | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    |
| 1                  | 5    | 6692 | 713  | 466  | 376  | 398  | 281  | 596  | 592  | 346  |
| 2                  | 7    | 9    | 4160 | 158  | 33   | 5    | 64   | 40   | 44   | 16   |
| 3                  | 224  | 5    | 260  | 3860 | 0    | 1808 | 33   | 5    | 1013 | 83   |
| 4                  | 44   | 6    | 191  | 169  | 3197 | 372  | 64   | 1806 | 178  | 2914 |
| 5                  | 1147 | 6    | 216  | 474  | 184  | 1585 | 1434 | 10   | 296  | 25   |
| 6                  | 4416 | 6    | 177  | 55   | 150  | 120  | 4019 | 24   | 74   | 49   |
| 7                  | 5    | 10   | 66   | 33   | 1891 | 170  | 1    | 3769 | 163  | 2444 |
| 8                  | 75   | 8    | 175  | 916  | 11   | 963  | 22   | 15   | 3491 | 72   |
| 9                  | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    |

# Cont'd

```

1 fviz_nbclust(
2   my_basket,
3   kmeans,
4   k.max = 25,
5   method = "wss",
6   diss = get_dist(my_basket, method = "spearman")
7 )

```

